

Async Programming in Python: Fundamentals

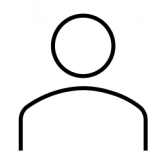
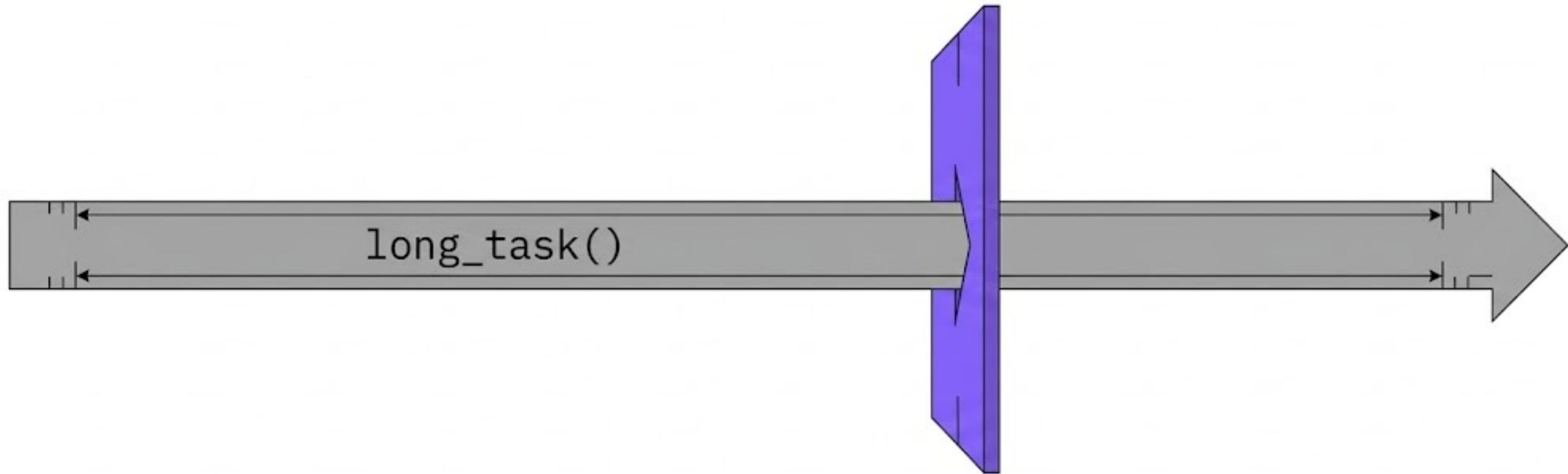
파이썬 비동기 프로그래밍 기초

비동기 심화와 유의사항

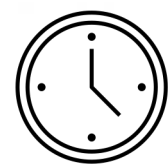
비동기 프로그래밍 심화



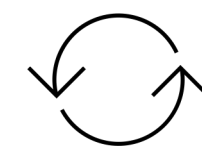
무조건 끝까지 실행하는 것이 정답은 아님



사용자가
화면을 이탈했을 때



설정된 대기 시간(Timeout)을
초과했을 때



더 최신 요청이 들어왔을 때

더 이상 필요 없는 작업은 시스템 자원 확보를 위해 즉시 중단해야 함



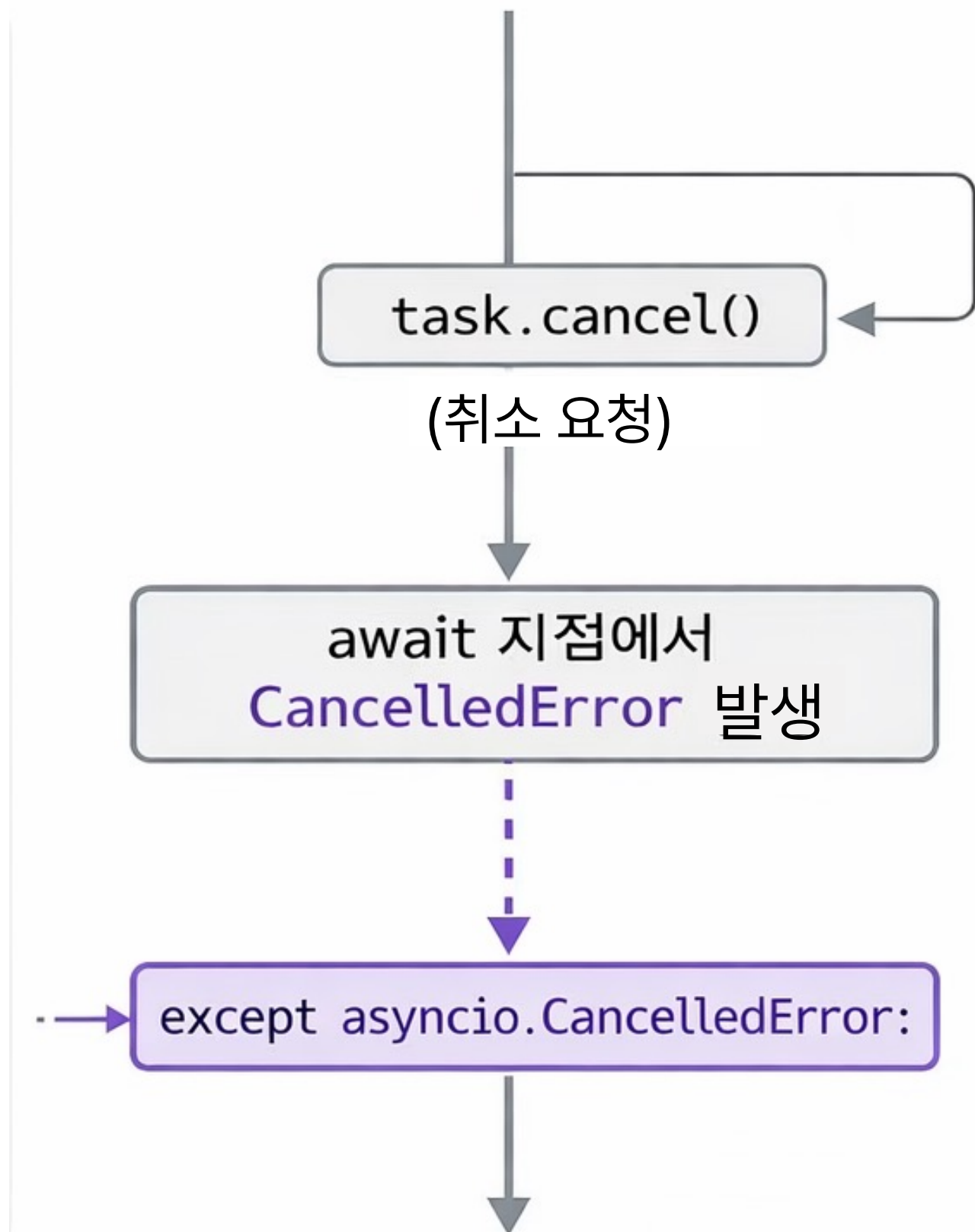
Cancel()과 CancelledError의 제어 흐름

```
import asyncio

async def long_task():
    try:
        print("작업 시작")
        await asyncio.sleep(10)
        print("작업 완료")
    except asyncio.CancelledError:
        print("작업이 취소되었습니다.")
        raise

async def main():
    task = asyncio.create_task(long_task())
    await asyncio.sleep(2) # 2초 대기
    task.cancel()
    try:
        await task
    except asyncio.CancelledError:
        print("메인: 태스크 취소 확인")

asyncio.run(main())
```





task.cancel() 문법 상세

```
async def long_task():
    try:
        await asyncio.sleep(10)
    except asyncio.CancelledError:
        print("취소됨! 정리 중...")

async def main():
    task = asyncio.create_task(long_task())
    await asyncio.sleep(1) # 1초 후
    task.cancel() # 취소 요청!
    try:
        await task
    except asyncio.CancelledError:
        print("확인 완료")
```

```
task.cancel(msg=None)
```

대상: **Task** 객체 (create_task의 반환값)

반환값: **True** (취소 요청 성공) / **False**

결과: 다음 **await**에서 **CancelledError** 발생

핵심 포인트

- cancel()은 **요청**일 뿐, 즉시 중단이 아님
- 다음 **await** 지점에서 CancelledError 발생
- try/except로 **정리 코드 실행** 가능

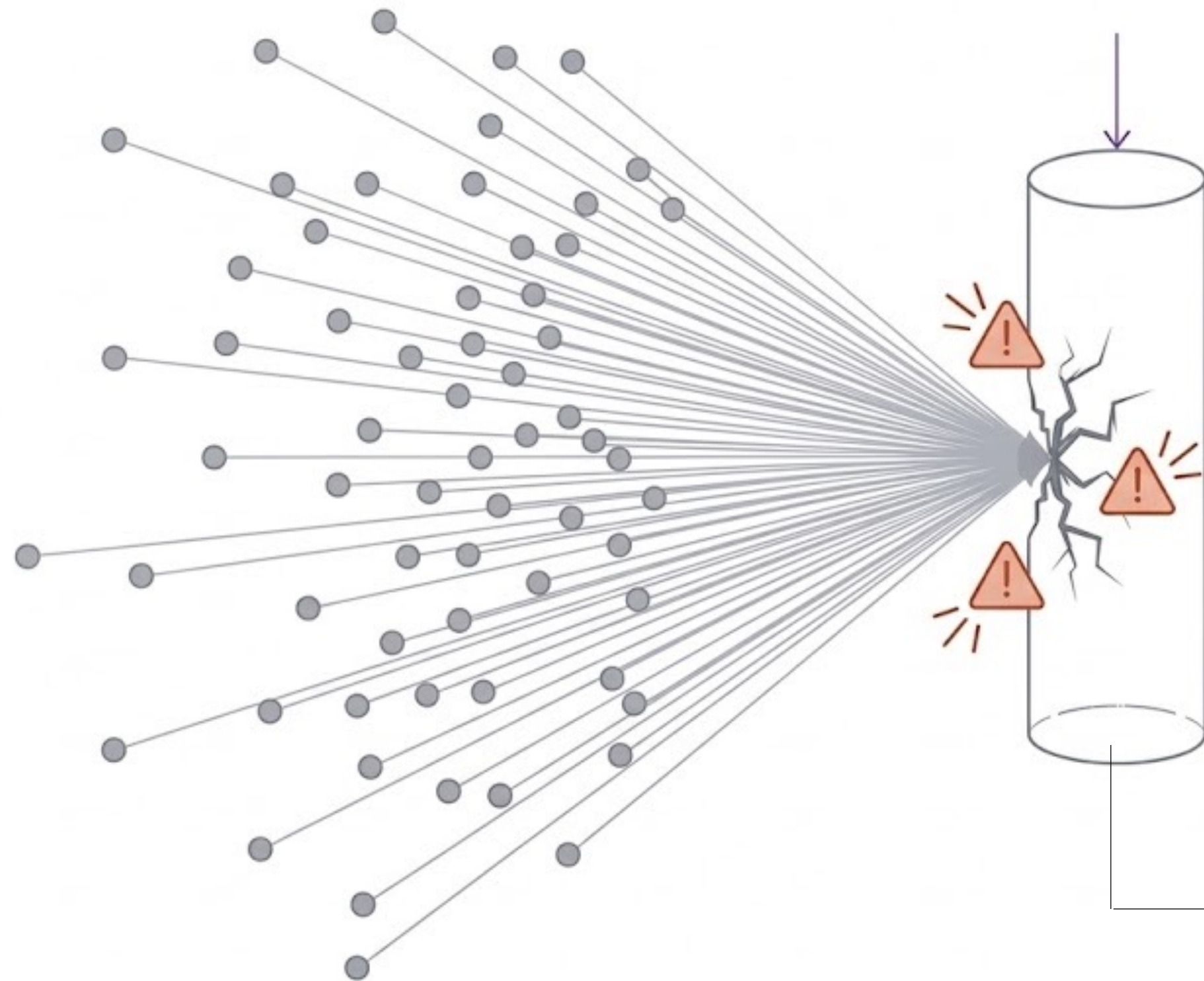


asyncio의 주요 에러 정리

에러	발생 상황	대처법
TimeoutError	wait_for()에서 시간 초과	try/except로 포착, 대체 로직 실행
CancelledError	task.cancel()로 태스크 취소	try/except 또는 finally에서 정리 작업
RuntimeError	이미 루프 실행 중에 asyncio.run() 호출	중첩 호출 금지, async 함수 안에서는 await 사용
ExceptionGroup	TaskGroup 내 여러 태스크에서 예외 발생	except* 문법으로 타입별 분리 처리 (3.11+)
RuntimeWarning	코루틴을 await 없이 그냥 호출	반드시 await 또는 create_task로 실행



무한한 동시성은 시스템의 파괴를 부름

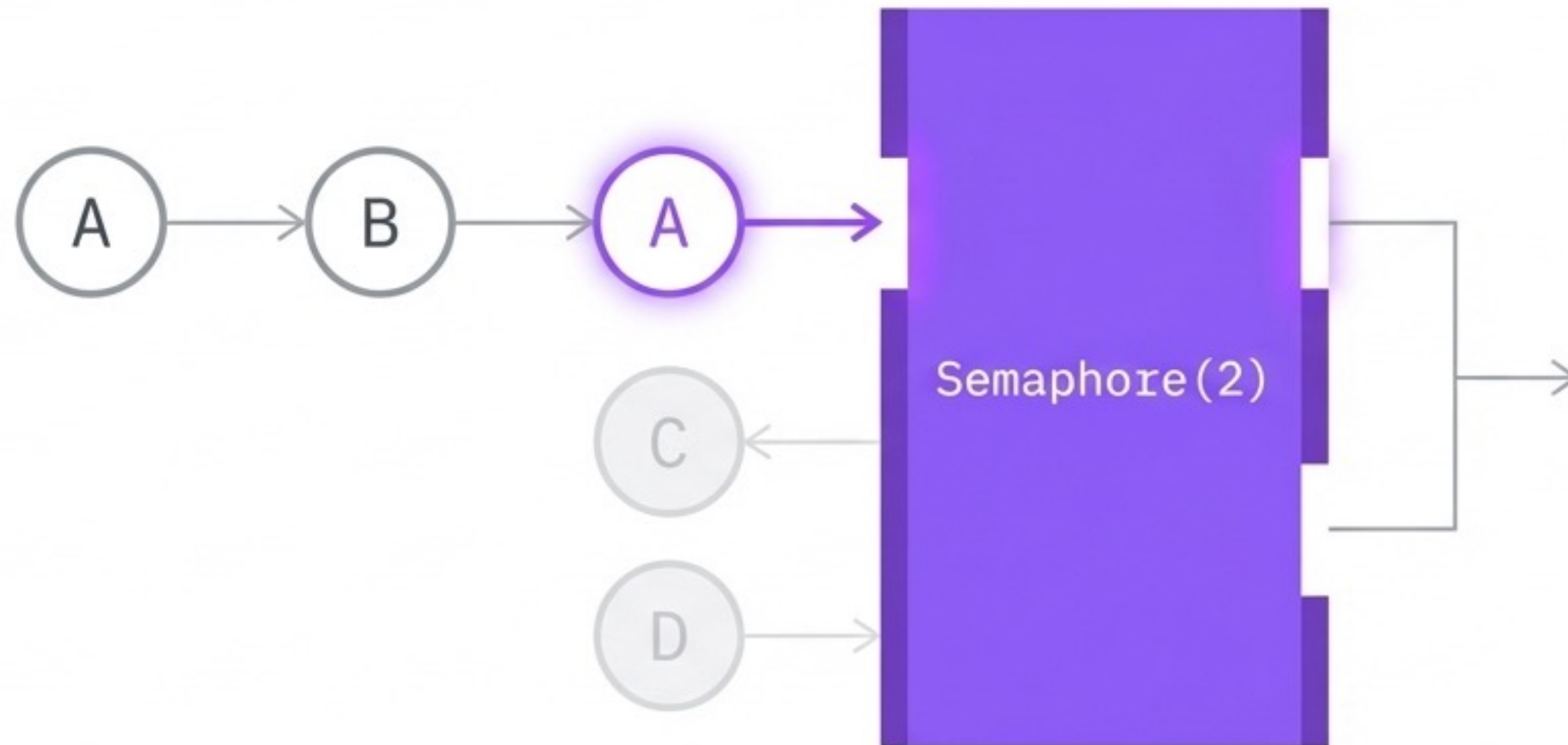


External API / Database

- 외부 API의 요청 횟수 제한 (Rate Limit) 초과
- 데이터베이스 커넥션 풀 고갈
- 시스템 메모리 및 CPU 자원 한계 도달



Semaphore : 허락된 만큼만 통과시키는 톨게이트



비동기의 장점은 살리되, 외부 서비스에 과도한 부담을 주지 않도록
동시 실행 수를 엄격하게 조절함



Semaphore : 동시 실행 수 제한하기

```
import asyncio

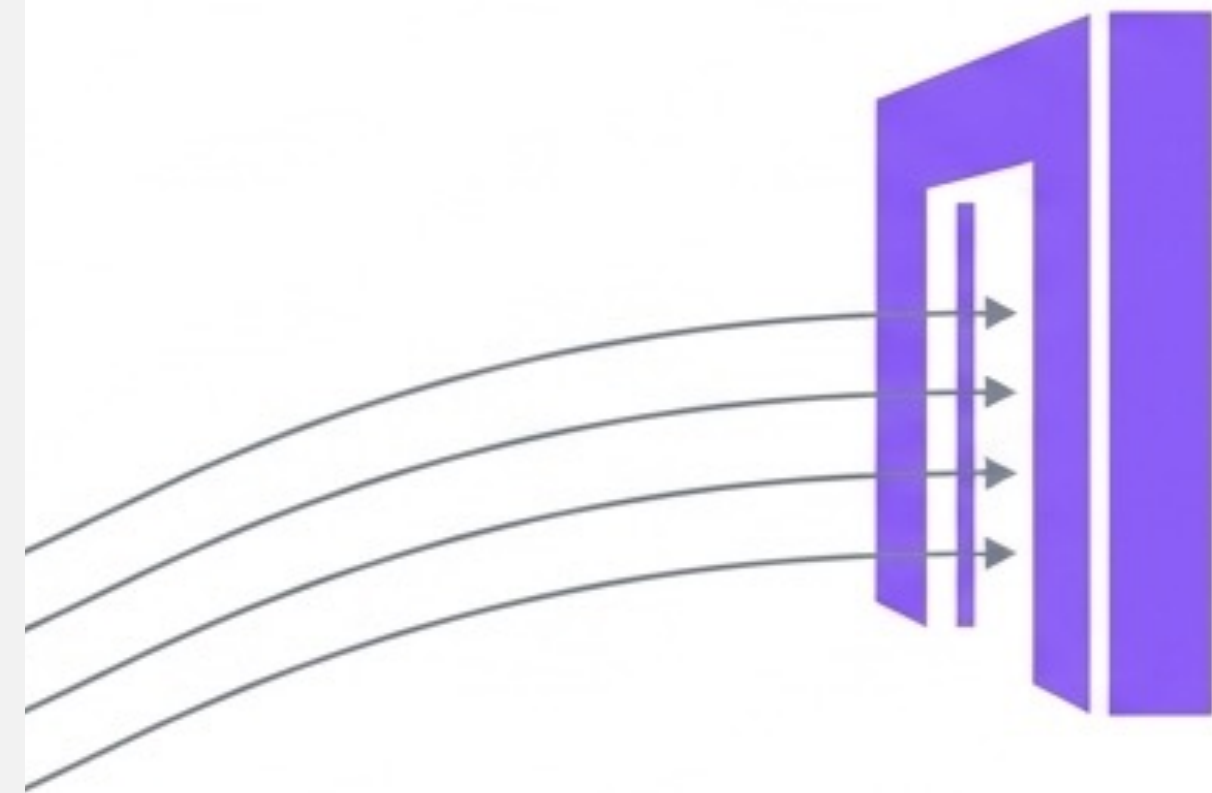
semaphore = asyncio.Semaphore(2)

async def worker(name: str, delay: int):
    async with semaphore:
        print(f"{name} 시작")
        await asyncio.sleep(delay)
        print(f"{name} 완료")

async def main():
    await asyncio.gather(
        worker("A", 2),
        worker("B", 2),
        worker("C", 2),
        worker("D", 2),
    )

asyncio.run(main())
```

Gather로 4개의 작업을 동시에 요청해도,
Semaphore(2) 블록에 의해 실제로는 한 번에 2개씩만 실행됨





asyncio.Semaphore 문법 상세

```
sem = asyncio.Semaphore(3) # 동시 3개만 허용

async def fetch(url):
    async with sem:
        # 이 안에서는 최대 3개만 동시 실행
        await asyncio.sleep(1)
        return f"{url} done"

async def main():
    urls = [f"api/{i}" for i in range(10)]
    await asyncio.gather(
        *[fetch(u) for u in urls])
# 10개 요청이지만 동시에 3개씩만 실행
```

```
sem = asyncio.Semaphore(최대 수)

async with sem:
```

인자: **동시 실행 최대 수** (int)

사용법: **async with sem:** 블록으로 진입

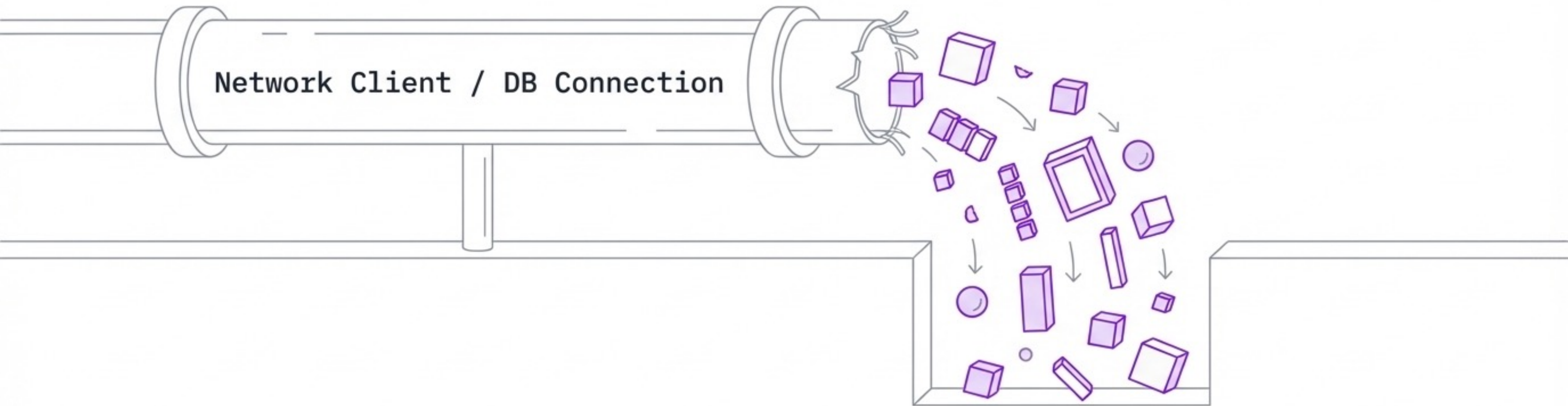
동작: **자리 있으면 진입**, 없으면 대기

핵심 포인트

- 동시 접속 수를 **제한**하여 서버 과부하 방지
- 블록 진입 = **acquire**, 블록 종료 = **release** 자동
- API 호출, DB 연결 등 **실무 필수**



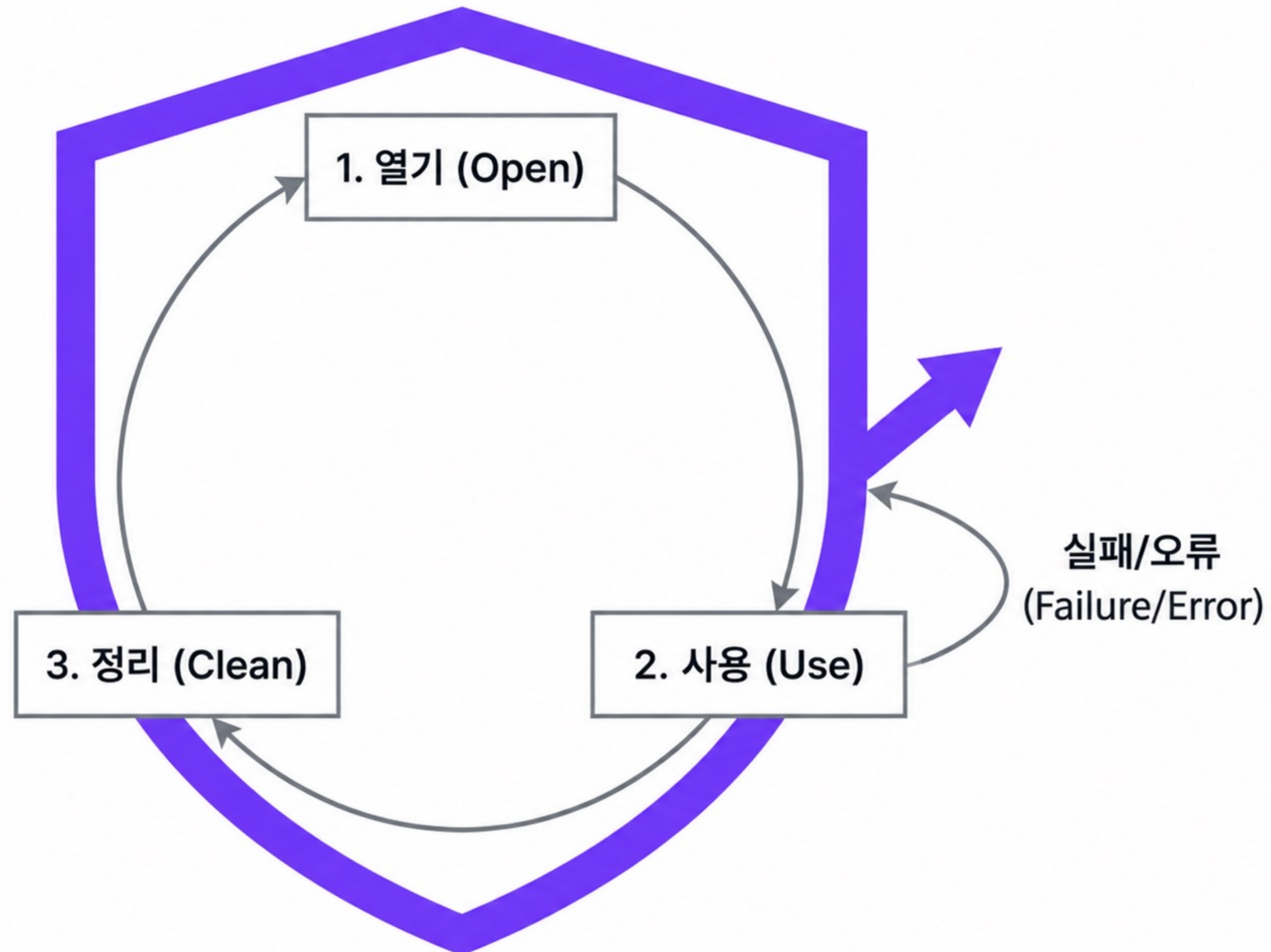
열어둔 문은 자원의 누수로 이어짐



비동기 코드에서 다루는 네트워크 클라이언트나 데이터베이스 연결 객체는 단순한 변수가 아닌 물리적 자원임. 비동기로 열었다면, 반드시 비동기로 닫아야 함



안전한 자원 관리를 위한 보호막



`async with`는 작업 중 에러가 발생하더라도
자원 정리(Cleanup) 단계를 반드시 보장하는 비동기 패턴



async with를 활용한 클라이언트 수명 주기 관리

```
import asyncio
```

```
import httpx
```

```
async def fetch_example():
```

```
    async with httpx.AsyncClient() as client:
```

```
        response = await client.get("https://example.com", timeout=5.0)
```

```
        print(response.status_code)
```

```
asyncio.run(fetch_example())
```

비동기 전용 HTTP 클라이언트 생성

응답이 올 때까지 이벤트 루프에
제어권 반환 및 5초 타임아웃 방어



비동기 프로그래밍 심화

상황 (Situation)	초보자의 접근 (Basic Async)	실무자의 제어 (Managed Async)
더 이상 필요 없는 작업	끝날 때까지 방치	<code>cancel()</code> 호출 및 롤백
10,000개의 API 요청	한 번에 모두 실행	<code>Semaphore</code> 로 동시성 제한
네트워크 연결 사용	열어두고 방치함	<code>async with</code> 로 자동 정리

비동기 프로그래밍의 유의사항



성공적인 비동기 설계를 위한 3가지 원칙



무조건적인 만능열쇠가 아니다

비동기는 모든 연산을 빠르게 만들지
않음. CPU 집중 작업이 아닌, 대기
시간이 발생하는 I/O 작업(네트워크,
파일)에 전략적으로 도입해야 함



전체 흐름의 일관성

함수 하나만 async를 붙인다고 해결되지
않음. 내부에서 사용하는 도구와
라이브러리, 그리고 기다림(await)까지
비동기 구조로 통일되어야 함



복잡해진 흐름과 검증

동시에 처리되는 작업은 흐름이 복잡해짐.
성공, 실패, 예외 상황에 대한 비동기 함수
테스트가 일반 코드보다 더욱 중요함



비동기 프로그래밍의 3가지 함정



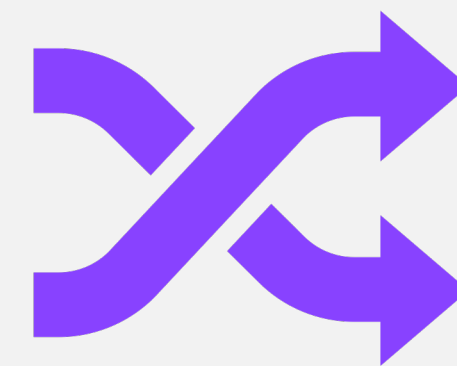
await 누락

코루틴 객체만
생성되고
실제로는 실행되지
않는 현상



동기 sleep 사용

이벤트 루프 전체를
멈춰버리는
가장 치명적인 실수



동기 라이브러리 혼용

겉모습만 비동기일 뿐,
실제로는 병목을
유발하는 구조



Await 누락 (The 'Forgotten Await' Problem)

경고: RuntimeWarning

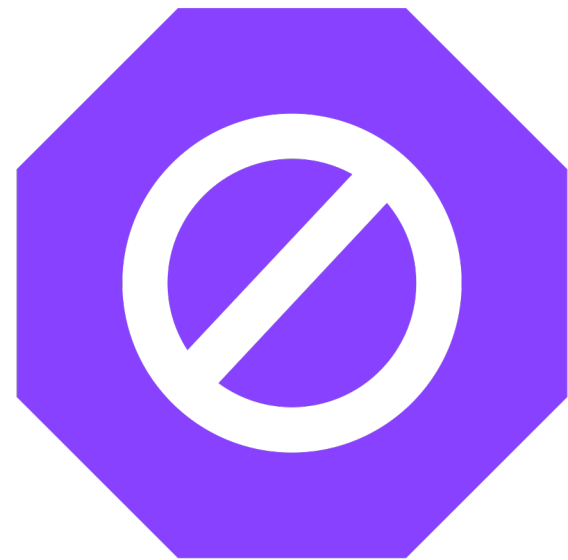
```
async def fetch_data():  
    ...  
result = fetch_data()
```

잊힌 `await`: 코루틴을 호출만 하고 `await`를 빼먹으면 코드는 실행되지 않음

`asyncio.run(main(), debug=True)`
모드를 켜면 쉽게 탐지할 수 있음



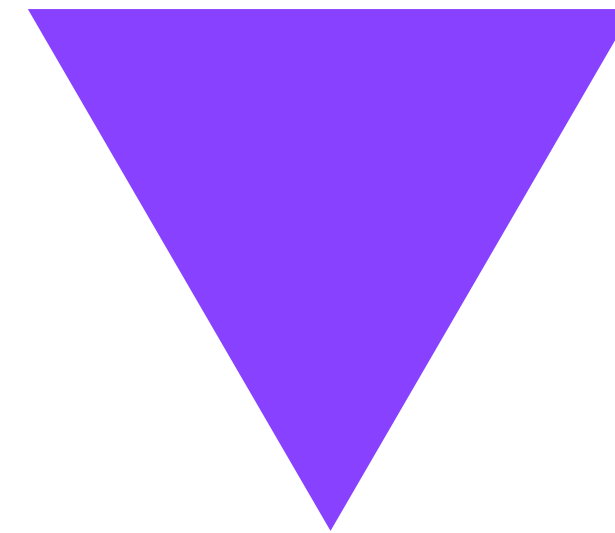
멈춤의 두 가지 방식 : `time.sleep` vs. `asyncio.sleep`



```
time.sleep(1)
```

시스템 전체 정지

현재 실행 흐름을 완전히 멈춤
비동기 환경에서 사용하면 전체 흐름이
막혀버림



```
await asyncio.sleep(1)
```

제어권 양보

'이 작업은 1초 기다릴 테니,
그동안 다른 작업을 처리하세요.'
기다림조차 비동기 방식으로 처리해야 함



동기식 블로킹 함수 대처

문제 상황: 동기식 블로킹 함수가 실행되면 이벤트 루프 전체가 멈춰버리는(Freezing) 치명적인 병목이 발생함

해결책: `run_in_executor`를 사용하여 블로킹 함수를 메인 루프가 아닌 별도의 '스레드 풀(Thread Pool)'로 격리하여 실행함





loop.run_in_executor() 문법 상세

```
import asyncio, time

def blocking_io():
    time.sleep(2) # 동기 블로킹!
    return "done"

async def main():
    loop = asyncio.get_running_loop()
    result = await loop.run_in_executor(
        None,          # 기본 스레드풀
        blocking_io    # 함수 참조 (()없이)
    )
    print(result)
```

```
await loop.run_in_executor(
    executor, 동기 함수)
```

인자①: **executor** (None = 기본 스레드풀)

인자②: **동기 함수 참조** (() 없이!)

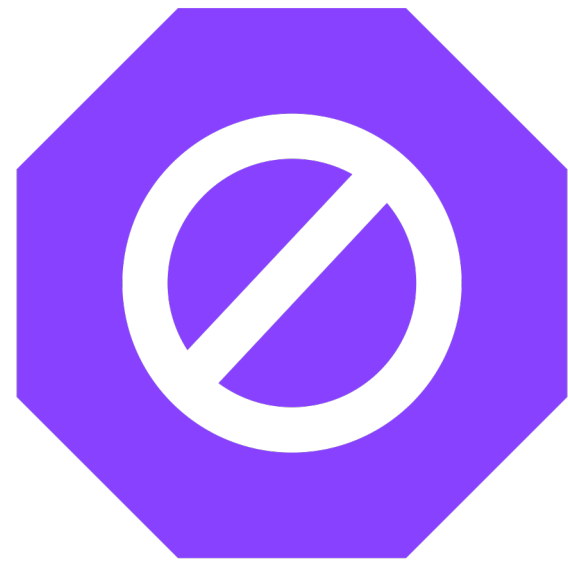
반환값: 동기 함수의 **실행 결과값**

핵심 포인트

- 동기 함수를 **별도 스레드**에서 실행 → 루프 블로킹 방지
- 비동기 전환이 안 되는 **레거시 코드 대처용**
- ProcessPoolExecutor 전달 시 **프로세스 풀**로도 가능



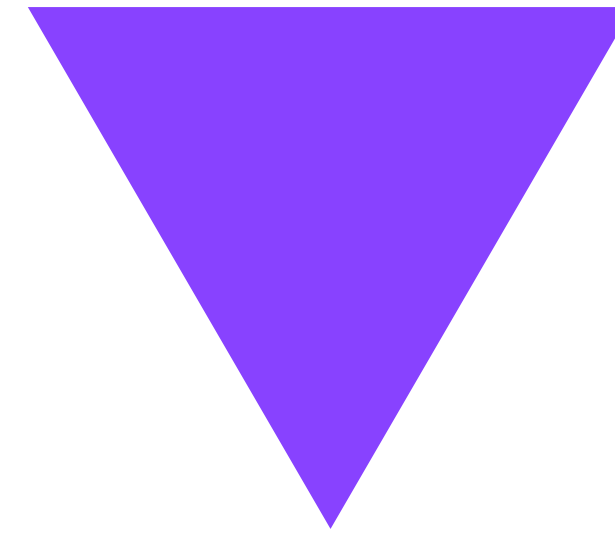
멈춤의 두 가지 방식 : `time.sleep` vs. `asyncio.sleep`



```
time.sleep(1)
```

시스템 전체 정지

현재 실행 흐름을 완전히 멈춤
비동기 환경에서 사용하면 전체 흐름이
막혀버림



```
await asyncio.sleep(1)
```

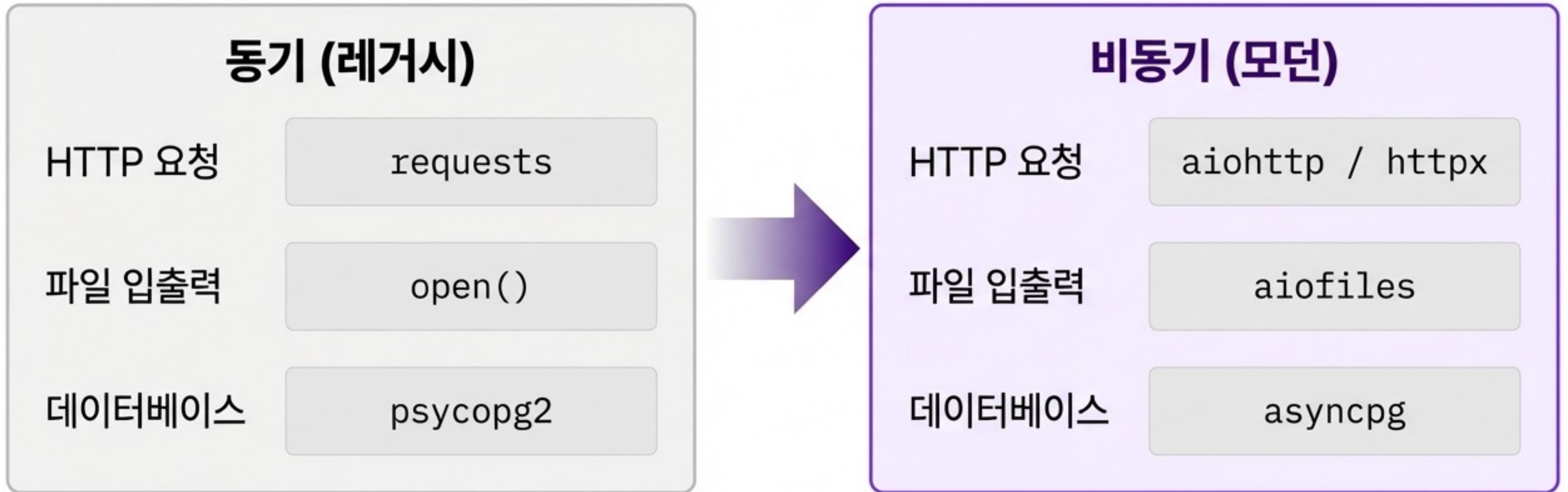
제어권 양보

'이 작업은 1초 기다릴 테니,
그동안 다른 작업을 처리하세요.'
기다림조차 비동기 방식으로 처리해야 함



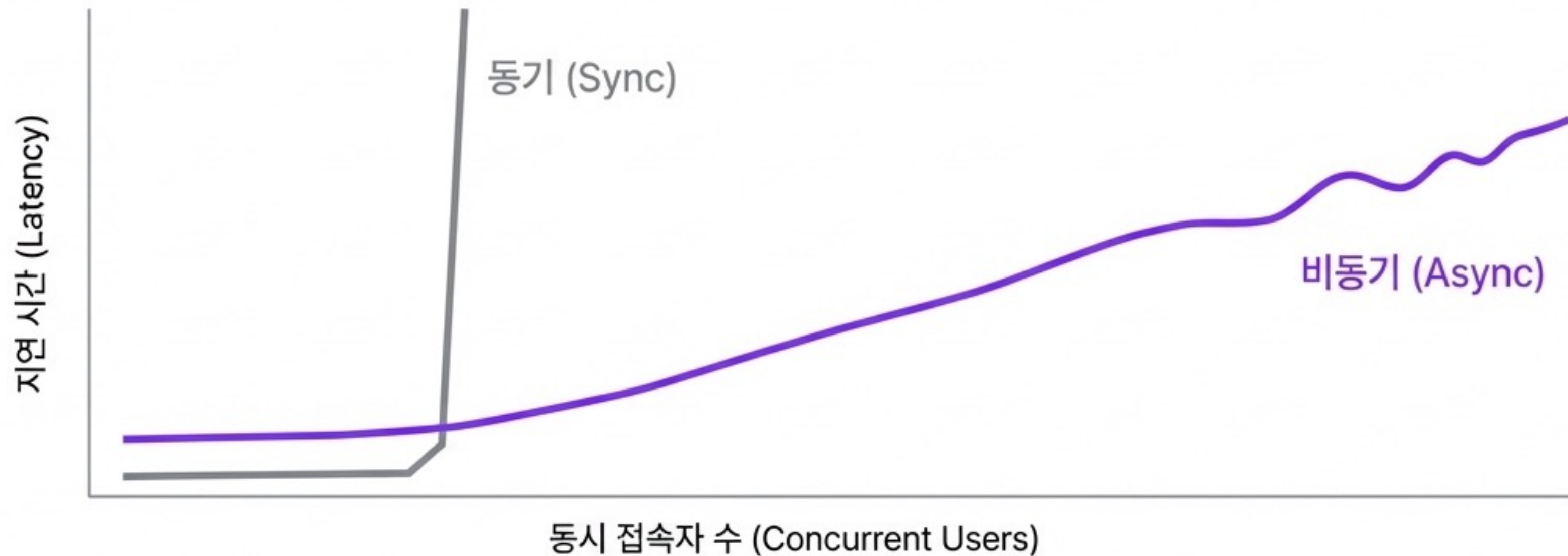
비동기를 위한 전용 생태계 전환

진정한 성능 향상을 위해서는 I/O가 발생하는 모든 지점(네트워크, 파일, DB)을
비동기 전용 라이브러리로 교체해야 함





비동기 성능의 진실과 오해



비동기는 무조건 빠른 것이 아님

단일 요청의 처리 속도를 높이는 것이 아니라, 동시에 처리할 수 있는 요청의 수(처리량, Throughput)를 극대화하는 기술
부하 상황에서는 지연 시간(Latency)의 변동성이 커질 수 있음을 설계 시 인지해야 함



'협력적 멀티태스킹'



파이썬 비동기 프로그래밍은 시스템이 강제로 작업을 나누는 것이 아니라,
개발자가 명시적으로 `await`를 통해 제어권을 양보하며
코드들끼리 서로 협력(Cooperative)하는 정교한 **오케스트레이션**